

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

docker compose

Orquestación local

Objetivos

Al finalizar esta parte vas a poder:

- **Declarar** un stack multicontenedor: servicios, redes, volúmenes
- **Parametrizar** con variables de entorno y manejar secretos sin hardcodearlos
- **Buildear** imágenes desde el propio Compose
- **Resolver** el orden de arranque con healthchecks y depends_on

docker compose

Permite construir aplicaciones utilizando un archivo YAML para indicar las acciones de:

- ejecutar uno o más contenedores
- definir la cantidad de instancias para cada contenedor
- crear y usar volúmenes
- crear y usar redes
- definir variables de ambiente
- definir qué pasa si un contenedor falla
- definir dependencia entre contenedores

docker compose implementa, de una forma más simple, el concepto de **orquestación** de contenedores.

Ejemplo: Contenedor simple

```
services:
  db:
    image: mysql
    restart: always
    environment:
      MYSQL_DATABASE: 'db'
      MYSQL_USER: 'user'
      MYSQL_PASSWORD: 'password'
      MYSQL_ROOT_PASSWORD: 'password'
    ports:
      - '3306:3306'
    expose:
      - '3306'
    volumes:
      - my-db:/var/lib/mysql
volumes:
  my-db:
```

Services • image • restart

```
services:
  db:
    image: mysql
    restart: always
    environment:
      MYSQL_DATABASE: 'db'
      MYSQL_USER: 'user'
      MYSQL_PASSWORD: 'password'
      MYSQL_ROOT_PASSWORD: 'password'
    ports:
      - '3306:3306'
    expose:
      - '3306'
    volumes:
      - my-db:/var/lib/mysql
volumes:
  my-db:
```

Services (contenedores)

Cada objeto dentro de esta clave representa un contenedor a ejecutar. En este ejemplo, db.

Image

Formato URL/`image:version`. Si se omite la URL, Docker Hub. Es la misma referencia que vimos en la primera parte.

Restart

Qué hace el orquestador cuando el contenedor no está en ejecución: `no`, `always`, `on-failure`, `unless-stopped`.

Environment

```
services:
  db:
    image: mysql
    restart: always
    environment:
      MYSQL_DATABASE: 'db'
      MYSQL_USER: 'user'
      MYSQL_PASSWORD: 'password'
      MYSQL_ROOT_PASSWORD: 'password'
    ports:
      - '3306:3306'
    expose:
      - '3306'
    volumes:
      - my-db:/var/lib/mysql
volumes:
  my-db:
```

Environment

Variables de ambiente creadas dentro del contenedor. Es la forma más simple de parametrizar su comportamiento; acá setean el usuario administrador de la base.

⚠ Passwords hardcodeadas en el YAML, que va a git. En un rato lo arreglamos.

Ports • expose • volumes

```
services:
  db:
    image: mysql
    restart: always
    environment:
      MYSQL_DATABASE: 'db'
      MYSQL_USER: 'user'
      MYSQL_PASSWORD: 'password'
      MYSQL_ROOT_PASSWORD: 'password'
    ports:
      - '33060:3306'
    expose:
      - '3306'
    volumes:
      - my-db:/var/lib/mysql
volumes:
  my-db:
```

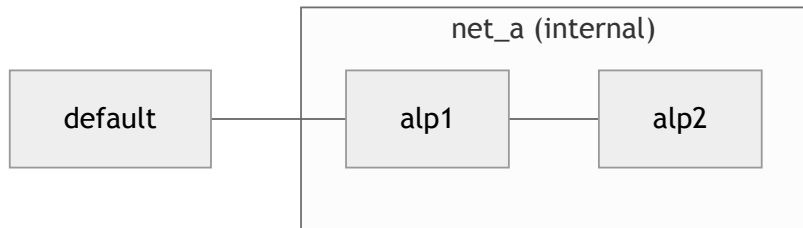
Ports – publica un puerto del contenedor en el **host** (host: contenedor). Sólo en modo bridge.

Expose – expone el puerto al resto de los contenedores de la red, sin publicarlo en el host.

Volumes – referencia (dentro del servicio) y declaración (global). Misma sintaxis que el `-v` de `docker run`.

Multi contenedor y redes

```
networks:
  net_a:
    internal: true
services:
  alp1:
    image: alpine
    networks:
      - default
      - net_a
    command: sleep infinity
  alp2:
    image: alpine
    networks:
      - net_a
    command: sleep infinity
```



Networks se declara global y se referencia por servicio. `internal: true` deshabilita la salida fuera del host.

Dentro de una red, los servicios se resuelven **por nombre**: alp1 llega a alp2 como `http://alp2`.

Variables de ambiente

```
services:
  db:
    image: mysql
    environment:
      MYSQL_DATABASE: ${DB_MYSQL_DB-db}
      MYSQL_USER: ${DB_MYSQL_USER-mysql}
      MYSQL_PASSWORD: ${DB_MYSQL_PASSWORD}
  app:
    image: myapp
    environment:
      - MYSQL_USER=${DB_MYSQL_USER-mysql}
      - MYSQL_PASSWORD=${DB_MYSQL_PASSWORD}
```

Dos formatos válidos: objeto ENV: valor o lista
ENV=valor. \${VAR-default} toma el default si no
está definida.

```
services:
  db:
    env_file:
      - .env1
      - .env2
    environment:
      - USER=fede
```

```
# .env1      # .env2
USER=carlos  PASSWD=pizza123
PASSWD=asado123 SOURCE=carbón
```

El container termina con:

```
USER=fede      ← environment gana
PASSWD=pizza123 ← .env2 pisa a .env1
SOURCE=carbón
```

Gestión de secretos

```
secrets:
  server-certificate:
    external: true
  db-password:
    file: ./secrets/db-password.txt

services:
  db:
    image: mysql
    restart: always
    secrets:
      - server-certificate
      - db-password
```

external: true – definido en el Docker engine, fuera de este compose. Compartible entre varios.

file: – se lee de un archivo relativo al compose (que **no** va a git).

Cada secreto referenciado se monta como archivo en:

/run/secrets/<nombre-del-secreto>

Montado hasta la destrucción del contenedor.

Building desde compose

Cada servicio puede buildear su propia imagen en vez de bajarla:

- `build`: ruta al **build context**
- `dockerfile`: nombre del Dockerfile si no es el default
- `args`: los ARG que vimos en la primera parte
- `target`: qué stage del multi-stage buildear

```
services:
  a:
    image: service_a
    build:
      context: ./service-a
      dockerfile: a.Dockerfile
      target: production
      args:
        NODE_VERSION: 20
  b:
    image: service_b
    build: ./service-b
```

```
docker compose build --no-cache
docker compose up --build
```

docker compose watch

Actualiza servicios automáticamente al guardar cambios. Sólo funciona con servicios que usan build.

```
services:
  web:
    build: .
    command: npm start
    develop:
      watch:
        - action: sync
          path: ./web
          target: /src/web
          ignore:
            - node_modules/
        - action: rebuild
          path: package.json
```

Acciones: sync (copia archivos), rebuild (reconstruye la imagen), sync+restart.

Orden de arranque

El problema de fondo de todo stack multicontenedor

La race condition

COMPOSE • STARTUP ORDER

Avanzar tiempo

Reset

t = 0 ms · fork

Sin healthcheck

depends_on: db

POSTGRES

starting



API

esperando

service_healthy

condition: service_healthy

POSTGRES

starting



API

esperando

Container **started** no es servicio **ready**. **depends_on** sin healthcheck conecta demasiado temprano.

Dos formas de resolverlo

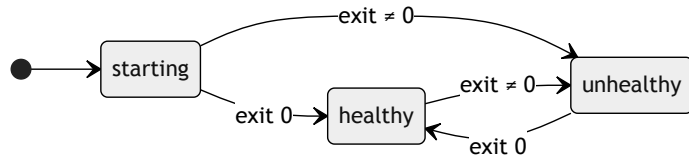
```
# ANTI-PATTERN: mete lógica de infra dentro de la imagen de la app
ENTRYPOINT ["/wait-for-it.sh", "db:5432", "--", "npm", "start"]
```

```
# FIX: healthcheck + depends_on condicional
services:
  core_database:
    image: postgres:18
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  backend_api:
    build: .
    depends_on:
      core_database:
        condition: service_healthy
```

Estados del healthcheck

Un comando que Docker ejecuta periódicamente **dentro** del contenedor para comprobar si está "sano".



Exit code de la prueba: **0** = healthy · cualquier otro = unhealthy.

No reinicia contenedores automáticamente; solo informa del estado.



Readiness vs liveness

Readiness – *¿puede aceptar trabajo?*

Si falla, los dependientes no deberían usarlo todavía.
En Compose, `service_healthy` los mantiene esperando.

Liveness – *¿sigue vivo, o hay que matarlo?*

Si falla, el workload está roto y la plataforma debe reiniciarlo.

El healthcheck de Compose está más cerca de **readiness**: no reinicia el container cuando los checks fallan.

¿Dónde se define el healthcheck?

Dos lugares posibles: en el Dockerfile (HEALTHCHECK ...) o en el docker-compose.yml (healthcheck:). Si el chequeo es intrínseco a la imagen, va en el Dockerfile; si depende del entorno, en Compose.

El campo test admite ["CMD", "exe", "arg"] (exec directo) y ["CMD-SHELL", "comando"] (pasa por sh -c).

```
# HTTP
["CMD-SHELL", "curl -fsS http://localhost:8080/health || exit 1"]

# Postgres
["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres}"]
```

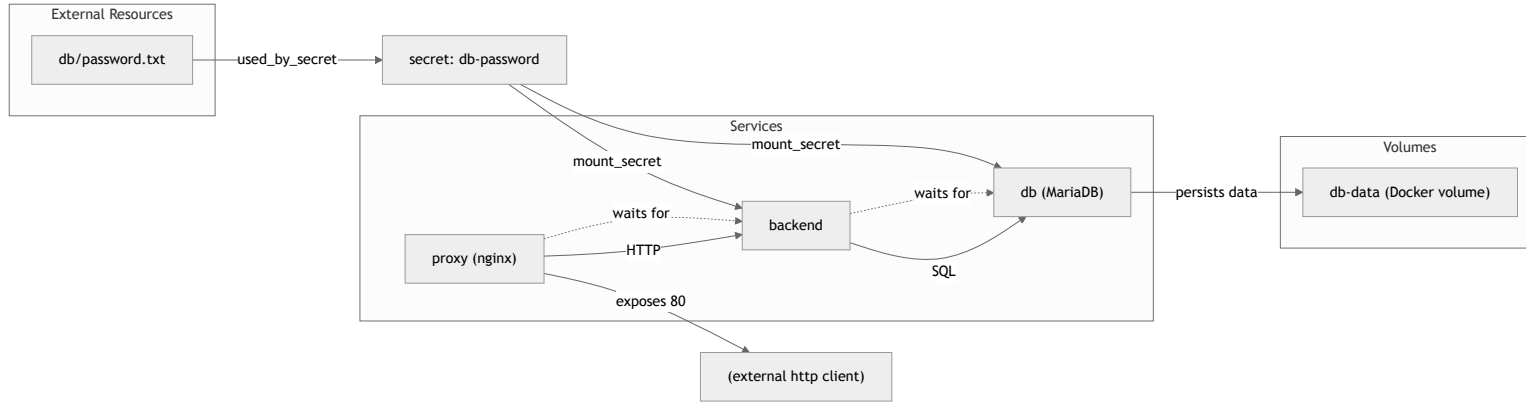
```
# Redis
["CMD", "redis-cli", "ping"]

# Mongo
["CMD-SHELL", "mongosh --quiet --eval 'db.runCommand({ping:1})'"]
```

Buenas prácticas: comandos rápidos y locales · start_period acorde al warm-up · sin red externa · las dependencias del test (curl, pg_isready) tienen que estar **en la imagen**.

Un stack completo

apigw + backend + mariadb



Fuente: <https://github.com/docker/awesome-compose/blob/master/nginx-golang-mysql/compose.yaml>

El servicio de base de datos

```
services:
  db:
    image: mariadb:10-focal
    command: '--default-authentication-plugin=mysql_native_password'
    restart: always
    healthcheck:
      test:
        - 'CMD-SHELL'
        - 'mysqladmin ping -h 127.0.0.1 --password="$$(cat /run/secrets/db-password)" --silent'
      interval: 3s
      retries: 5
      start_period: 30s
    secrets:
      - db-password
    volumes:
      - db-data:/var/lib/mysql
    environment:
      - MYSQL_DATABASE=example
      - MYSQL_ROOT_PASSWORD_FILE=/run/secrets/db-password
    expose:
      - 3306
```

Más abajo, las declaraciones globales: `volumes: db-data:` y `secrets: db-password: {file: db/password.txt}`.

Fuente: <https://github.com/docker/awesome-compose/blob/master/nginx-golang-mysql/compose.yaml>

El backend espera a la base

```
services:
  backend:
    build:
      context: backend
      target: builder
    secrets:
      - db-password
    depends_on:
      db:
        condition: service_healthy

  proxy:
    image: nginx
    volumes:
      - type: bind
        source: ./proxy/nginx.conf
        target: /etc/nginx/conf.d/default.conf
        read_only: true
    ports:
      - 80:80
    depends_on:
      - backend
```

Fuente: <https://github.com/docker/awesome-compose/blob/master/nginx-golang-mysql/compose.yaml>

Para seguir

- **Compose file reference** – docs.docker.com/compose/compose-file
- **Startup order y healthchecks** – docs.docker.com/compose/how-tos/startup-order
- **Ejemplos oficiales** – docs.docker.com/reference/samples
- **awesome-compose** – github.com/docker/awesome-compose

A la práctica

Stack multicontenedor: App + DB